



ASIC Verification
EE5213
Exercise 2 Report
FSM Reachability Analysis

Student: Nguyen Thai Thanh Binh - 2570175

Instructor: TS. Tran Hoang Linh

April 1, 2026

1 Exercise 2: FSM Reachability Analysis

1.1 Objective

Implement the State Reachability Analysis algorithm for an arbitrary Finite State Machine (FSM) using C++. The program computes the set of reachable states given a set of initial states and a transition relation.

1.2 Design Specifications

- **Input:** Set of initial states (I), and a transition relation (R) mapping current states to next possible states.
- **Output:** The complete set of reachable states (ARS - All Reached States).
- **Algorithm Strategy:**
 - Track **All Reached States** (ARS) and **New Next States** (NNS or Frontier Set).
 - Iteratively compute the next states (NS) from the current frontier NNS .
 - Update the frontier as newly discovered states ($NNS = NS - ARS$).
 - Add the new frontier to the all reached states ($ARS = ARS \cup NNS$).
 - Terminate when the frontier is empty.

1.3 Example FSM Analysis

To illustrate the algorithm, consider the FSM defined in the implementation with the following transition relations:

- $S_0 \rightarrow \{S_1, S_5\}$
- $S_1 \rightarrow \{S_2\}$
- $S_5 \rightarrow \{S_5, S_2\}$
- $S_2 \rightarrow \{S_3\}$
- $S_3 \rightarrow \{S_4\}$
- $S_4 \rightarrow \emptyset$

The initial state set is $I = \{S_0\}$. The state diagram is shown in Figure 1.

Step-by-Step Execution:

1. Initialization:

- $ARS = \{S_0\}$
- Frontier $NNS = \{S_0\}$

2. Iteration 1:

- Compute next states from NNS : $NS = \{S_1, S_5\}$

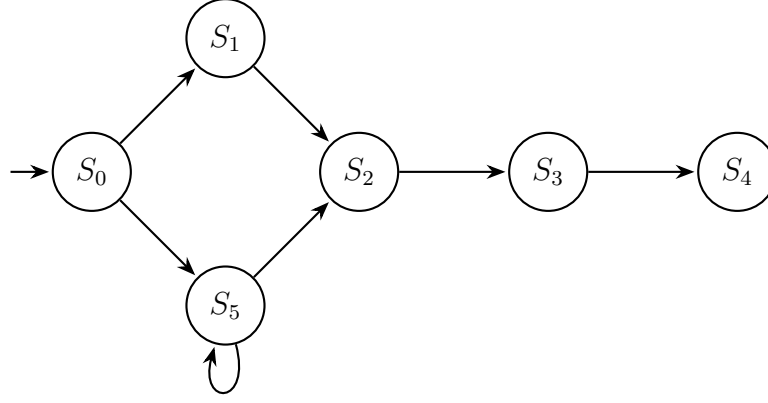


Figure 1: State Diagram of the Example FSM

- Compute new frontier: $NNS = NS \setminus ARS = \{S_1, S_5\} \setminus \{S_0\} = \{S_1, S_5\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_5\}$

3. Iteration 2:

- Compute next states from NNS : $NS = \{S_2, S_5\}$ (from $S_1 \rightarrow S_2$ and $S_5 \rightarrow S_2, S_5$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_2, S_5\} \setminus \{S_0, S_1, S_5\} = \{S_2\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_5\}$

4. Iteration 3:

- Compute next states from NNS : $NS = \{S_3\}$ (from $S_2 \rightarrow S_3$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_3\} \setminus \{S_0, S_1, S_2, S_5\} = \{S_3\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_5\}$

5. Iteration 4:

- Compute next states from NNS : $NS = \{S_4\}$ (from $S_3 \rightarrow S_4$)
- Compute new frontier: $NNS = NS \setminus ARS = \{S_4\} \setminus \{S_0, S_1, S_2, S_3, S_5\} = \{S_4\}$
- Update ARS : $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

6. Iteration 5:

- Compute next states from NNS : $NS = \emptyset$ (from $S_4 \rightarrow \emptyset$)
- Compute new frontier: $NNS = NS \setminus ARS = \emptyset \setminus ARS = \emptyset$
- The frontier NNS is empty, so the algorithm terminates.

Final Result: $ARS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

1.4 Implementation (fsm_reachability.cpp)

```
1 #include <iostream>
2 #include <vector>
3 #include <set>
4 #include <map>
5 #include <algorithm>
6
7 typedef int State;
8
9 void print_set(const std::string& name, const std::set<State>& s) {
10     std::cout << name << " = { ";
11     for (State val : s) std::cout << "S" << val << " ";
12     std::cout << "}\n";
13 }
14
15 std::set<State> Reachability(
16     const std::map<State, std::set<State>>& R,
17     const std::set<State>& I
18 ) {
19     std::set<State> ARS = I; // All Reached States
20     std::set<State> NNS = I; // New Next States (Frontier Set)
21     int iteration = 1;
22
23     std::cout << "--- Initialization ---\n";
24     print_set("ARS", ARS);
25     print_set("NNS", NNS);
26
27     while (true) {
28         std::cout << "\n--- Iteration " << iteration++ << " ---\n";
29         std::set<State> NS;
30         for (State s : NNS) {
31             if (R.find(s) != R.end()) {
32                 for (State next_s : R.at(s)) {
33                     NS.insert(next_s);
34                 }
35             }
36         }
37         print_set("NS ", NS);
38
39         std::set<State> new_NNS;
40         std::set_difference(
41             NS.begin(), NS.end(),
42             ARS.begin(), ARS.end(),
43             std::inserter(new_NNS, new_NNS.begin())
44         );
45         NNS = new_NNS;
46         print_set("NNS", NNS);
47
48         if (NNS.empty()) {
49             std::cout << "NNS is empty. Terminating.\n";
50             break;
51         }
52         ARS.insert(NNS.begin(), NNS.end());
53         print_set("ARS", ARS);
54     }
55     return ARS;
56 }
```

```

57
58 int main() {
59     std::map<State, std::set<State>> transition_relation = {
60         {0, {1, 5}}, {1, {2}}, {5, {5, 2}},
61         {2, {3}}, {3, {4}}, {4, {}}
62     };
63     std::set<State> initial_states = {0};
64
65     std::cout << "Starting Reachability Analysis...\n\n";
66     std::set<State> reachable_states = Reachability(transition_relation
67 , initial_states);
68
69     std::cout << "\nFinal Reachable States (ARS): { ";
70     for (State s : reachable_states) std::cout << "S" << s << " ";
71     std::cout << "}\n";
72     return 0;
73 }

```

1.5 Execution Results

The screenshot shows the OneCompiler interface. On the left, the C++ code for the Reachability Analysis is displayed. On the right, the execution output is shown, including the program's output and the execution time (4 ms) and memory usage (44 MB).

```

Main.cpp
1 #include <iostream>
2 #include <vector>
3 #include <set>
4 #include <map>
5 #include <algorithm>
6
7 typedef int State;
8
9 void print_set(const std::string& name, const std::set<State>& s) {
10     std::cout << name << " = { ";
11     for (State val : s) std::cout << "S" << val << " ";
12     std::cout << "}\n";
13 }
14
15 std::set<State> Reachability(
16     const std::map<State, std::set<State>>& R,
17     const std::set<State>& I
18 ) {
19     std::set<State> ARS = I; // ALL Reached States
20     std::set<State> NNS = I; // New Next States (Frontier Set)
21     int iteration = 1;
22
23     std::cout << "---- Initialization ----\n";
24     print_set("ARS", ARS);
25     print_set("NNS", NNS);
26
27     while (true) {
28         std::cout << "\n--- Iteration " << iteration++ << " ---\n";
29         std::set<State> NS;
30         for (State s : NNS) {
31             if (R.find(s) != R.end()) {
32                 for (State next_s : R.at(s)) {
33                     NS.insert(next_s);
34                 }
35             }
36         }
37         print_set("NS ", NS);
38
39         std::set<State> new_NNS;
40         std::set_difference(
41             NS.begin(), NS.end(),
42             ARS.begin(), ARS.end(),
43             std::inserter(new_NNS, new_NNS.begin())
44         );
45         NNS = new_NNS;
46         print_set("NNS", NNS);
47
48         if (NNS.empty()) {
49             std::cout << "NNS is empty. Terminating.\n";

```

```

STOIN
Input for the program (Optional)

Output
Starting Reachability Analysis...
---- Initialization ----
ARS = { S0 }
NNS = { S0 }

--- Iteration 1 ---
NS = { S1 S5 }
NNS = { S1 S5 }
ARS = { S0 S1 S5 }

--- Iteration 2 ---
NS = { S2 S5 }
NNS = { S2 }
ARS = { S0 S1 S2 S5 }

--- Iteration 3 ---
NS = { S3 }
NNS = { S3 }
ARS = { S0 S1 S2 S3 S5 }

--- Iteration 4 ---
NS = { S4 }
NNS = { S4 }
ARS = { S0 S1 S2 S3 S4 S5 }

--- Iteration 5 ---
NS = { }
NNS = { }
NNS is empty. Terminating.

Final Reachable States (ARS): { S0 S1 S2 S3 S4 S5 }

```

Figure 2: Program Execution Output

```

1 Starting Reachability Analysis...
2
3 --- Initialization ---
4 ARS = { S0 }
5 NNS = { S0 }
6
7 --- Iteration 1 ---
8 NS = { S1 S5 }
9 NNS = { S1 S5 }
10 ARS = { S0 S1 S5 }

```

```
11
12 --- Iteration 2 ---
13 NS  = { S2 S5 }
14 NNS = { S2 }
15 ARS = { S0 S1 S2 S5 }
16
17 --- Iteration 3 ---
18 NS  = { S3 }
19 NNS = { S3 }
20 ARS = { S0 S1 S2 S3 S5 }
21
22 --- Iteration 4 ---
23 NS  = { S4 }
24 NNS = { S4 }
25 ARS = { S0 S1 S2 S3 S4 S5 }
26
27 --- Iteration 5 ---
28 NS  = { }
29 NNS = { }
30 NNS is empty. Terminating.
31
32 Final Reachable States (ARS): { S0 S1 S2 S3 S4 S5 }
```